Lesson: Logistic Regression, Decision Trees, and k-NN

Introdcution

Imagine you're designing an app that predicts whether a person will buy a product based on age and income, or building a system to classify handwritten digits. In both cases, choosing the right model is critical.

In this lesson, we dive deep into three foundational classification models—Logistic Regression, Decision Trees, and k-Nearest Neighbors (k-NN)—to understand how they work, where they shine, and where they might fall short.

Learning Outcomes

By the end of this lesson, you will be able to:

- Understand how logistic regression, decision trees, and k-NN work.
- Describe the assumptions and decision boundaries of each model.
- Evaluate and compare the strengths and weaknesses of these models.

Logistic Regression, Decision Trees, and k-NN

Logistic regression, decision trees, and k-nearest neighbors (k-NN) are foundational algorithms in supervised learning. Each takes a different approach to solving classification problems and provides unique advantages depending on the dataset and context. In the following sections, we will explore how each of these algorithms works, where they are most effective, and how to apply them in practice.

1. Logistic Regression

Logistic regression is used when the output is categorical, most often binary (e.g., yes/no, 0/1). Despite the name, it's a classification algorithm, not a regression one. A log of the odds is used as the dependent variable. Using a logit function, logistic regression predicts the probability that a binary event will occur.

> Imagine betting: if the odds of success are 4:1, then the log of the odds helps you map that into a continuous scale.

How it works:

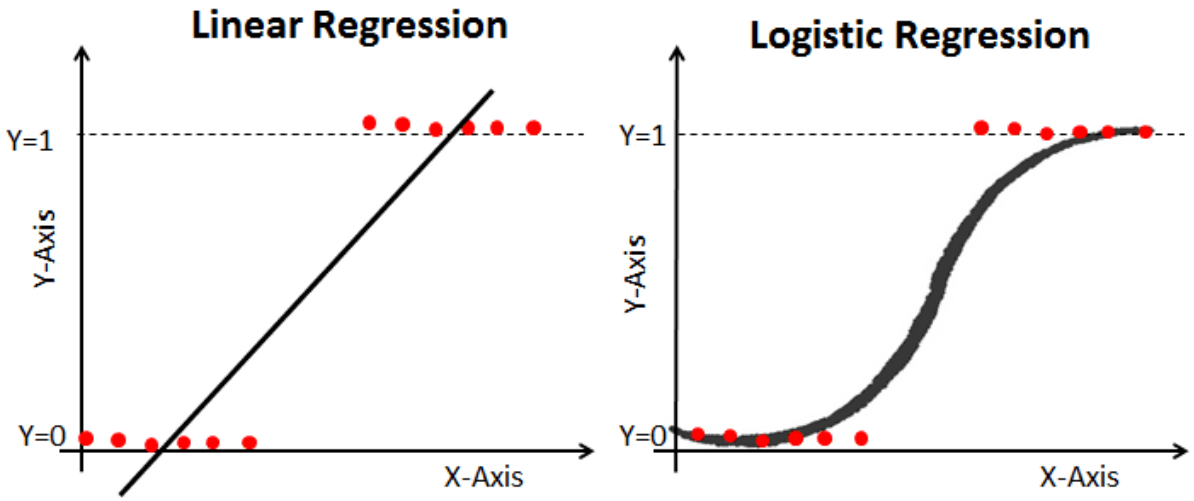
Logistic regression models the probability that a given input belongs to a particular class using the sigmoid function:

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n)}}$$

Where:

- P(Y=1) = Probability of the data point belonging to the positive class
- X1 ,... ,Xn = Input Features
- b0,....,bn = Input weights that the algorithm learns during training

This sigmoid curve shows how logistic regression maps inputs to probabilities between 0 and 1.



Key Concepts:

1. Decision Boundary: A linear boundary defined by the logistic function's threshold (usually 0.5).
2. Assumption: Assumes linear separability in the feature space.
3. Output: Probability of class membership (can be converted to class label).

Types of Logistic Regression

- **Binary Logistic Regression**: The target variable has only two possible outcomes such as Spam or Not Spam, Cancer or No Cancer.
- **Multinomial Logistic Regression**: The target variable has three or more nominal categories, such as predicting the type of Wine.
- **Ordinal Logistic Regression**: the target variable has three or more ordinal categories, such as restaurant or product rating from 1 to 5.

Code Snippet:

```
# import the class
from sklearn.linear_model import LogisticRegression

# instantiate the model (using the default parameters)
logreg = LogisticRegression(random_state=16)

# fit the model with data
logreg.fit(X_train, y_train)

y_pred = logreg.predict(X_test)
```

Pros and Cons

The table below outlines some common strengths and weaknesses of this algorithm. Understanding these trade-offs can help determine when it is an appropriate choice for a given problem.

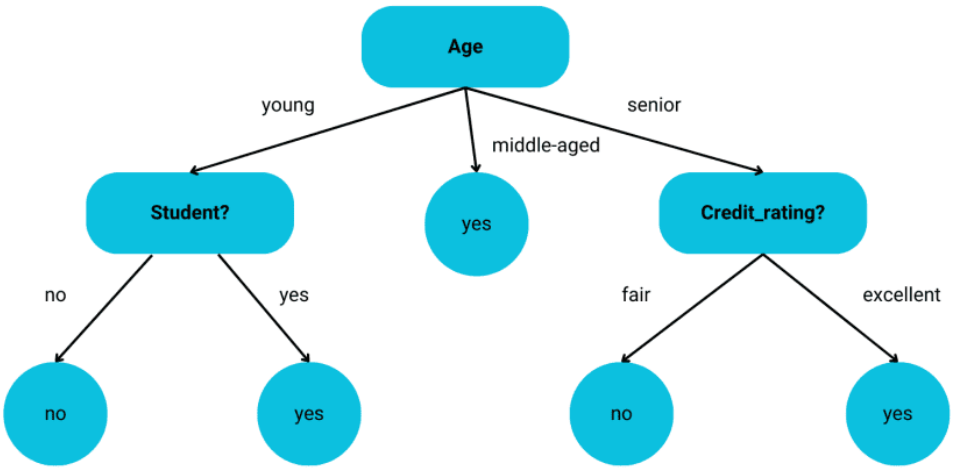
Strengths	**Weaknesses**
Simple, easy to understand and interpret	May underperform if the true relationship is nonlinear or if irrelevant features are included.
Well-calibrated for output probabilities	Experiences difficulty capturing complex relationships

2. Decision Trees

Decision tree algorithms are a type of probabilistic, tree-like structural model that continuously separates data to categorize or make predictions based on the results of previous questions. The model analyzes the data and provides answers to these questions to help you make more informed choices.

How it works:

- Start with the entire dataset at the root node.
- Select the best feature to split the data (based on measures like Gini impurity).
- Create child nodes for each possible value of the selected feature.
- Repeat steps 2–3 for each child node until a stopping criterion is met (e.g., maximum depth reached, minimum samples per leaf, or pure leaf nodes).
- Assign the majority class to each leaf node.



Key Concepts:

1. Decision Boundary: Axis-aligned, piecewise boundaries.
2. Assumption: No assumption of linearity or distribution.
3. Overfitting Risk: Trees can perfectly fit training data unless pruned.

Code Snippet:

```
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(max_depth=4)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Pros and Cons:

The table below summarizes the main strengths and weaknesses of decision trees. These points can help you evaluate when this algorithm is a suitable choice.

Strengths	Weaknesses
Easy to visualize and explain	Prone to overfitting (without pruning or depth limit)
Handles non-linear relationships	Sensitive to small data changes (instability)
No feature scaling required	Less accurate alone (compared to ensemble methods)

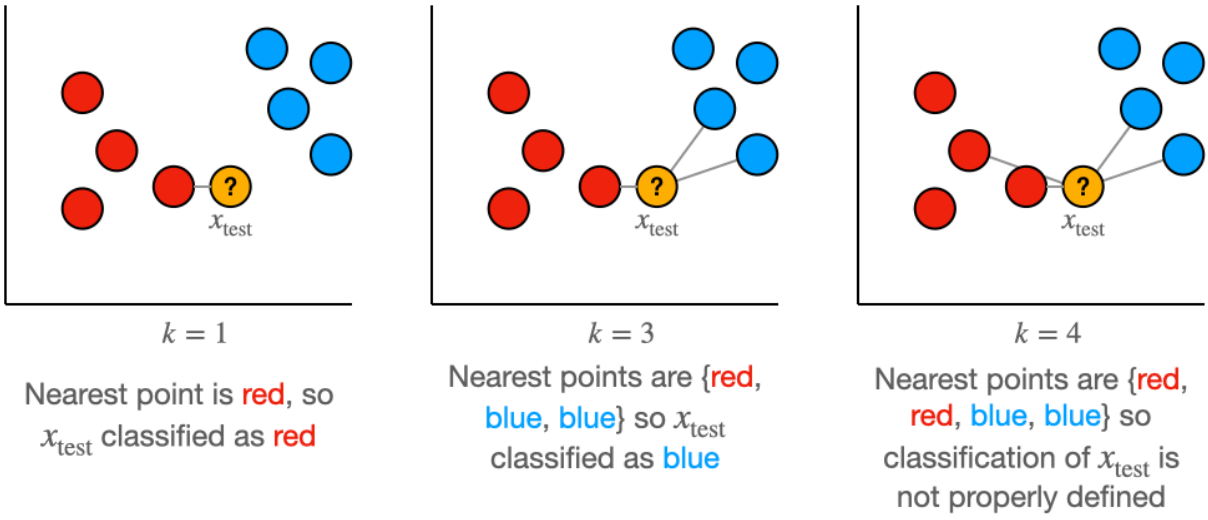
3. K-Nearest Neighbors

K-Nearest Neighbors is a statistical method that evaluates the proximity of one data point to another data point in order to decide whether or not the two data points can be grouped together.

The proximity of the data points represents the degree to which they are comparable to one another.

How it works:

- Compute distance (typically Euclidean) to all training samples.
- Take a majority vote (classification) or average (regression) of the k nearest points.



Key Concepts:

1. Decision Boundary: Highly non-linear and flexible, shaped by training data.
2. Assumption: Similar instances exist in proximity.
3. Hyperparameter: k (number of neighbors) is crucial for performance.

Code Snippet:

```
from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Pros and Cons:

The following table highlights the key strengths and limitations of the k-nearest neighbors (k-NN) algorithm. These points can guide its appropriate use depending on the data and task.

Strengths	Weaknesses
Simple and intuitive	Slow with large datasets (stores entire training set)
No training time	Requires feature scaling (e.g., standardization)
Flexible decision boundary	Impacted by irrelevant features or noisy data

What is Decision Boundary?

Decision boundary is the surface that separates different classes in a classification problem. It is a line (in 2D), a plane (in 3D), or a hyperplane (in higher dimensions) that defines the point at which the model decides to switch from predicting one class to another. The position and shape of this boundary are influenced by the learning algorithm, the data it has been trained on, and the features of that data.

Formal Definition:

For a binary classification problem, a decision boundary can be represented mathematically as the set of points x for which the model assigns equal probability to both classes. In logistic regression, for instance, the decision boundary occurs where the predicted probability of the positive class is 0.5. Mathematically, this can be expressed as:

$$P(y = 1 \mid x) = 0.5$$

Where x represents the feature vector, and y=1 indicates the positive class. The decision boundary is the locus of all points where the model is indifferent between the two classes, and any slight change in the feature vector x would push the prediction to one class or the other.

Types of Decision Boundaries

Decision boundaries define how a model separates different classes in the feature space. The shape of these boundaries depends on the algorithm used and the structure of the data.

Linear Decision Boundaries

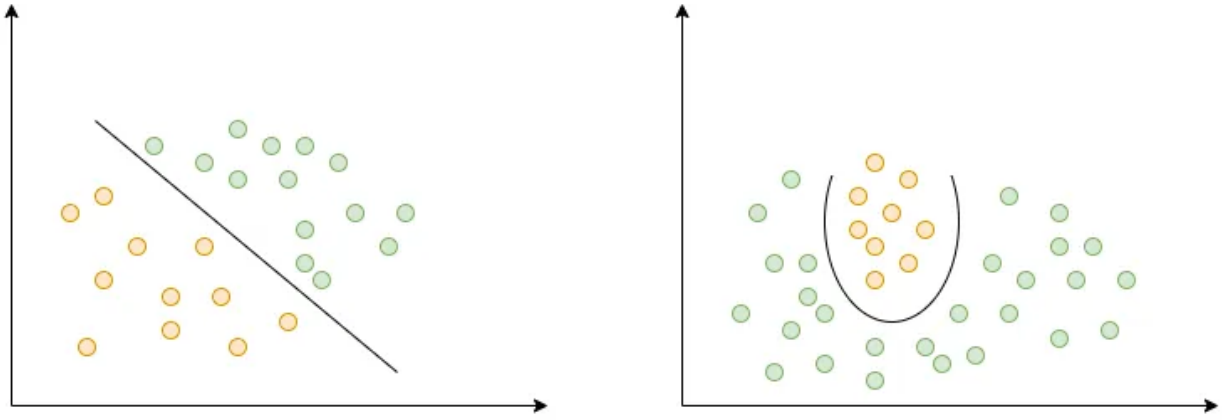
- Represented by straight lines in two dimensions, planes in three dimensions, or hyperplanes in higher dimensions
- Used by models such as logistic regression and linear support vector machines (SVMs)
- Most effective when the data is linearly separable

Example: Separating cats from dogs based on features like ear shape and tail length, where a straight line can divide the classes.

Nonlinear Decision Boundaries

- More flexible, curved, or complex boundaries formed by models like neural networks, decision trees, or SVMs with radial basis function (RBF) kernels
- Required when classes cannot be separated by a straight line

Example: Classifying healthy versus cancerous cells in medical images, where patterns in the data may follow complex, nonlinear distributions.



Impact of Decision Boundaries on Model Performance

The shape and placement of a model's decision boundary play a major role in how well it performs. To improve accuracy and reduce mistakes like overfitting or underfitting, it's important to understand how the boundary interacts with the data.

1. Overfitting and Complex Decision Boundaries

Overfitting happens when a model's decision boundary becomes too detailed, fitting even the smallest patterns or noise in the training data. Instead of learning the general trends, the model tries to memorize every point.

For example, a deep neural network with many layers might create a highly curved, nonlinear boundary that separates the training data perfectly—but fails on new, unseen data. The model performs well on training but struggles to generalize.

2. Underfitting and Simple Decision Boundaries

Underfitting occurs when a model's decision boundary is too basic to capture the actual patterns in the data. For example, a straight-line (linear) boundary might not work well in situations where the classes are separated in more complex ways.

Real-life example: In image classification, a linear model may fail to tell apart digits like "3" and "8" because it can't understand the curved shapes and details.

Model	Decision Boundary	Key Strengths	Key Weaknesses
Logistic Regression	Linear	Simple, interpretable; outputs well-calibrated probabilities	Assumes linear separability; struggles with complex relationships
Decision Trees	Axis-aligned, non-linear (stepwise)	Handles non-linear data; easy to visualize; no scaling needed	Prone to overfitting; unstable to data noise

Model	Decision Boundary	Key Strengths	Key Weaknesses
k-NN	Highly non-linear, data-driven	Intuitive; no training phase; flexible boundaries	Slow with large datasets; needs feature scaling; sensitive to noise

Conclusion

Choosing the right classification model depends heavily on the data characteristics and the problem you're solving:

1. Use Logistic Regression when relationships are linear and interpretability is important.
2. Choose Decision Trees for their flexibility and ease of explanation, especially with non-linear data.
3. Opt for k-NN when simplicity and decision boundary flexibility are needed, but be mindful of computational cost and noise.

Understanding the shape and behavior of each model’s decision boundary helps you match the model to your data and avoid common pitfalls like overfitting or underfitting